

Lab 2

ECE 455 / ECE 555, Fall 2026

Assigned	Wednesday, September 9
Due	Friday, September 18, 11:59 p.m. , through Canvas

Objective

- To set up the built-in SysTick timer
- To set up the PLL in the Tiva LaunchPad
- To explore the usage of interrupts in microcontrollers
- Abstract level system design using a Finite State Machine (FSM)

Teamwork

Laboratory work is done in teams of two, formed in the first week and kept for the whole semester. Only one lab report is required for each lab team. But both members must be able to explain every part of the submission to the TA. Every lab in this course is demonstrated in person to the TA during TA office hours. Bring your board, your circuit, and your results. The TA checks the operation, verifies that it meets the assigned functionality for each individual student. Both team members demonstrate, and either one may be asked to explain any part of the submission. TA office hours are posted on Canvas. Plan your demonstration in advance rather than in the last session before the due date.

Preparation

- You will need a Tiva LaunchPad and a PC with the Keil μ Vision IDE installed
- A breadboard for circuit connections
- At least 3 LEDs, 2 push-button switches, at least 3 resistors, and a few connecting wires
- The LEDs should be Red, Yellow, and Green

In the previous lab, we showed how to read and write the information to the data register of Port F, which is a pre-built interface on the Tiva LaunchPad. Further, we explored the concept of using bit-specific addressing and friendly program writing. In this lab, we will first explore the concept of interfacing an external switch and LEDs to the microcontroller. In addition to that, we will set up the inbuilt SysTick timer, the PLL, and interrupts. Finally, we will explore the concept of Finite State Machine programming, to design the system at the abstract layer. Again, your code must use bit-specific addressing and must follow friendly programming practices.

Part 1. Component Interfacing and Initialization Routine (20 points)

We will use three LEDs and two push-button switches connected at PA2 (Red), PA3 (Yellow), PA4 (Green) and PD1 (SW1), PD2 (SW2) respectively. The LEDs will be connected as positive logic and the switches will be connected as negative logic. In addition to the LEDs, you will need some resistors.

In this part, you should submit:

- The circuit diagram interfacing the LEDs and switches. The circuit diagram should be drawn neat and clean. We do not expect you to use any software, however, the drawing should be in compliance with basic Electrical Engineering practice. Drawing wire connections without a straightedge, or that do not look straight, will not serve the purpose. Also check the symbols for the LEDs, switches, resistors, ground, and the direction of the current. Submit an actual photograph of your prototype. **R1**
- The initialization function at Port A and Port D. The code should be friendly.

Note: Find the corresponding values of the resistors.

Part 2. Port Interfacing, Delay using SysTick, Configuring the PLL (40 points)

The SysTick timer is inbuilt, and part of any ARM based microcontroller. SysTick is helpful to create a precise time delay based on a number of clock ticks. You need to initialize SysTick, create the delay functions specified below (assume a 16 MHz clock frequency), and write code that demonstrates the usage of the above functions.

2A. Configuring the SysTick (20 points)

In this part, you should submit:

- SysTick_Init(), the initialization function for the SysTick timer
- SysTick_Delay100us_16MHz(), the function to create a 100 μ s delay
- SysTick_Delay1s_16MHz(), the function to create a 1 second delay using SysTick_Delay100us_16MHz()
- A code that demonstrates:
 - No switch pressed: toggle the Green LED
 - SW1 pressed: toggle the Red LED every 1 second, other LEDs off
 - SW2 pressed: blink a pattern (Red on, Yellow on, Green on, repeat)
 - Both switches pressed: flash the RYG on, RYG off

Note: The delay between each "on" should be 1 second. Only one LED should blink at a time.

2B. Configuring the PLL, and Configuring SysTick to Generate Interrupts (20 points)

In this part, you should submit:

- PLL_Init(), which configures the microcontroller to get the clock at 25 MHz from the PLL
- SysTick_Init_Interrupts(), the function to configure SysTick to generate an interrupt every 10 ms, timed against the 25 MHz PLL clock configured above and not the 16 MHz reset clock
- A code that demonstrates:
 - SW1 pressed: a binary up counter that counts from 0 to 7 (that is, 000, 001, ..., 111)
 - SW2 pressed: a binary down counter that counts from 7 to 0 (that is, 111, 110, ..., 000)
 - Other cases: Red LED (1 second on, 1 second off), Yellow LED (1 second on, 2 seconds off), and Green LED (1 second on, 3 seconds off)

Part 3. System Design at the Abstract Layer using an FSM (40 points)

An FSM is a framework that helps to solve the system design problem in an easier and cleaner way. In Embedded C, we define an FSM at certain basic abstract building blocks and simplified interconnections between them to carry out a function.

3A. Traffic Light Controller (15 points)

In class we designed a traffic light controller for the intersection of two one-way streets, North and East, as a Moore finite state machine. In this part you build that same controller on your board. The version in class drives six lights on two signal heads and reads two car sensors. Your board has three LEDs and two switches, so exactly two things change.

The lights. PA2 (Red), PA3 (Yellow) and PA4 (Green) are the North signal head. You do not build the East head. Take each state's output from the six-bit pattern on the state graph and keep the three North bits.

The sensors. SW1 is CarN and SW2 is CarE. A pressed switch means a car is waiting on that road. Build the input value the way we did in class, CarN as bit 1 and CarE as bit 0, so the four columns 00, 01, 10 and 11 of the state table keep the meanings they had on the slide.

Change only the two wait times. Use 3 seconds where the lecture used 30, and 1 second where it used 5, so the whole machine can be watched in a few seconds.

In this part, you should submit:

- The code for the traffic light controller.
- The state table must be data. Changing a state time, or an arrow in the graph, must be a change to the table and to nothing else.

Note: Use the functions created in Part 2 (PLL at 25 MHz, SysTick_Init() and the delay functions). There is no need to use the interrupt.

Your TA will check the following at the demonstration:

- With neither switch pressed, the light holds green.
- Press and hold SW2, a car on East. The light turns yellow for one second, then red.
- With the light red, press and hold SW2 only. It holds red.
- With the light red, press and hold SW1, a car on North. Green returns within four seconds.

3B. Sequence Detector Design (25 points)

In this part we will design a Moore based "1011" sequence detector, with and without overlapping (two separate programs).

Your output should look as follows as demonstrated to TA:

Press	1	2	3	4	5	6	7	8	9	10
Input	1	0	1	0	1	1	0	1	1	0
Without	0	0	0	0	0	1	0	0	0	0
With	0	0	0	0	0	1	0	0	1	0

In this example we will use SW1 and SW2 as connected in the previous parts. You can remove the yellow LED connections and keep the other two LEDs. We will use SW1 to input 0 and SW2 to input 1. The machine should advance only when either of the switches is pressed. When the sequence is not detected, the Red LED should remain on and the Green LED off. When the sequence is detected, the Green LED should be turned on and the Red LED off.

In this part you should submit the following:

- A State-Transition Graph for both the with-overlapping and without-overlapping sequence detectors. You should also summarize your design choices and any decisions made. **R2**
- Submit the code for the Moore based sequence detector (without overlapping)
- Submit the code for the Moore based sequence detector (with overlapping)

Report

- R1 and R2 will come only in the report. You are free to add information that can help the TA in grading, however, we do not expect any extra credit for writing background theory information. This should be submitted as Report2.pdf inside the .zip file.
- The one submission report should display the names of both students and contributions of each student.
- A single Keil µVision project folder with all code.

- Code should be commented for better understanding and to highlight any design choices or calculations that you made.
- The code should be properly structured and should make sense. Poorly structured code, though working, may severely affect your score.
- In Embedded Systems, it is preferred that your code should be as simple as possible so that the translation into binary code is shorter in size and portable. Writing unnecessary compiler-specific directives is not a good idea.
- The report and the Keil μ Vision project folder should be submitted in a single .zip folder.

AI Use Statement

Generative AI tools are permitted on this lab, but their use must be disclosed. End your report with a short AI Use Statement naming the tool, what you used it for, and what it contributed. If you used no AI tool, write "No generative AI tools were used."

Late Work

Submissions are due at 11:59 p.m. on the stated date. **Late submissions are not accepted.** If you must miss a submission date, inform the instructor in advance, or submit a legitimate, documented reason for the absence. Failure to do so results in a grade of zero for this assignment. See the syllabus for the full policy.